*NLP 2 · M2 IA — revue d'avancement J3*

# Que nous disent les logs HDFS ?

Repérer les blocs qui tournent mal, puis expliquer pourquoi : deux approches, une même équipe.


**TeamAI**

Alain Evina · Nassim Mezzai · Bachar Ben Slama · Linda Boukhit


*HDFS.log — un bloc, quatre lignes*

```
1 081110 010451 33 INFO dfs.FSNamesystem: BLOCK* N
  ameSystem.allocateBlock: /user/root/randtxt/_tem
  porary/_task_200811092030_0003_m_000269_0/part-…
2 081110 010451 5105 INFO dfs.DataNode$DataXceive
  r: Receiving block blk_-1013481023655112960 src:
  /10.251.74.134:45275 dest: /10.251.74.134:50010
3 081110 010451 5109 INFO dfs.DataNode$DataXceive
  r: Receiving block blk_-1013481023655112960 src:
  /10.251.74.134:48604 dest: /10.251.74.134:50010
4 081110 010454 5105 INFO dfs.DataNode$DataXceive
  r: writeBlock blk_-1013481023655112960 received
  exception java.io.IOException: Could not read f…
```

*là, ça casse. mais pourquoi ?*

*Le point de départ*

# Des centaines de milliers de blocs, et quelques pannes discrètes

**575 061**

blocs HDFS annotés

*Loghub HDFS_v1, logs de 2008*

**2,93 %**

d'anomalies seulement

*16 838 blocs : classes très déséquilibrées*

**29**

types d'événements

*de E1 à E29*

*Un label « anomalie » dit qu'il y a un problème… jamais pourquoi.*

C'est toute la raison d'être du volet B.

*Notre réponse*

# Deux façons de lire les mêmes logs

VOLET A

## Le guetteur

*« Ce bloc est-il normal ? »*

- Un BiLSTM **entraîné par nous**, exporté en ONNX
- Il tourne **dans le navigateur**, sans serveur
- Site statique, coût nul (GitHub Pages)
- Rapide, léger, entièrement maîtrisé

*les logs bruts*

VOLET B

## L'explicateur

*« Que s'est-il passé, et sur quoi je m'appuie ? »*

- SmolLM2-1.7B, un modèle ouvert, **en local**
- Adapté par LoRA sur des exemples annotés
- RAG sur la documentation officielle HDFS
- Faits, hypothèses, sources… ou « je ne sais pas »

Sept contrats d'échange (C1 à C7) fixent ce que chaque partie transmet aux autres : traces, vocabulaire, modèle, découpage, réponses, passages, évaluations.

*Qui fait quoi*

# Quatre personnes, un seul dépôt

## AE

### Alain Evina

*SLM et adaptation*

- ✓ Première inférence de SmolLM2
- ✓ Version 4 bits du modèle
- ✓ Tests et intégration continue
- ✓ Essai LoRA (en cours)

## NM

### Nassim Mezzai

*Données*

- ✓ Audit du corpus HDFS
- ✓ Parser et découpage reproductible
- ✓ Baseline de référence
- ✓ Format des logs du site

## BB

### Bachar Ben Slama

*Volet A et site*

- ✓ Export ONNX validé
- ✓ BiLSTM v1 entraîné
- ✓ Site v0 avec modèle en cache

## LB

### Linda Boukhit

*RAG et organisation*

- ✓ Corpus documentaire HDFS
- ✓ Prétraitement JS identique au Python
- ✓ Grille d'annotation (en relecture)
- ✓ Classeur partagé de l'équipe

*Les données*

# Un seul découpage, et on ne triche pas




### Tiré bloc par bloc

Découpage stratifié (seed 42) : chaque partie garde 2,93 % d'anomalies. Tous les modèles de l'équipe utilisent exactement le même.

### *Seulement 18 373 séquences distinctes !*

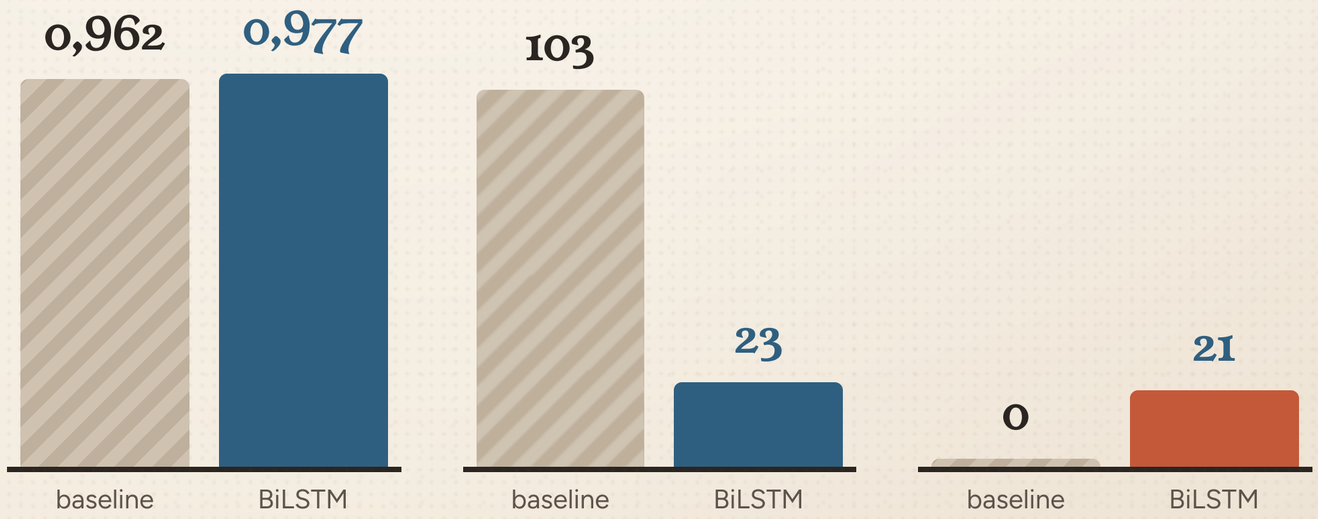
Beaucoup de blocs se ressemblent. On donne donc toujours deux scores : sur tout l'ensemble, et sur les **séquences jamais vues**, qui mesurent la vraie généralisation.

### Le test reste fermé

Aucun réglage, seuil ou exemple n'en est tiré : la règle est vérifiée automatiquement à chaque pull request.

*Volet A · premiers résultats*

# Le BiLSTM fait mieux que la baseline, à un prix près




**Macro-F1 sur les séquences jamais vues**
*meilleure généralisation*

**Fausses alertes (validation)**
*4,5× moins*

**Anomalies manquées (validation)**
*le prix à payer*

**0,996**
macro-F1 sur les 86 259 blocs de validation

**$1{,}2\times10^{-7}$**
écart maximal PyTorch / ONNX, 0 classement changé

**100 %**
des 575 061 blocs prétraités à l'identique en JavaScript et en Python

*Seuil choisi sur la validation : 0,99, au bord de la grille testée → à réexaminer.*

Mesures sur la validation. Le test officiel n'a pas encore été ouvert.

*Volet B · premiers pas*

# Le modèle parle déjà… mais pas encore juste

1

### Première inférence

SmolLM2-1.7B sur CPU : moins d'un token par seconde. Il ignore les WARN et invente des détails.

2

### Version 4 bits

Environ trois fois plus rapide, 2,8 Go de mémoire au lieu de 16.

3

### Corpus RAG

51 passages de la doc HDFS. 19 événements n'ont aucun passage qui les explique.

4

### Grille d'annotation

Une note sur 8 et trois exemples de calibration, dont une abstention.

5

### Essai LoRA

Chaîne validée sur CPU (2 pas, adaptateur rechargé). Les 30 pas se lanceront sur GPU, en 4 bits.

Observations sur une seule trace : ce n'est pas encore une évaluation.

*Notre façon de travailler*

# Un ticket, une branche, une relecture

Ticket → Branche → Pull request → **Relecture croisée** → dev → main · site en ligne

**20**

pull requests fusionnées, chacune reliée à son ticket

### Tests à chaque PR

Formats des contrats, docstrings, parité JavaScript / Python, test jamais utilisé pour régler.

### Un journal honnête

Chaque mesure garde sa commande, sa machine et ses limites. L'usage de l'IA y est tracé.

*Où on en est*

# Le point, sans maquillage

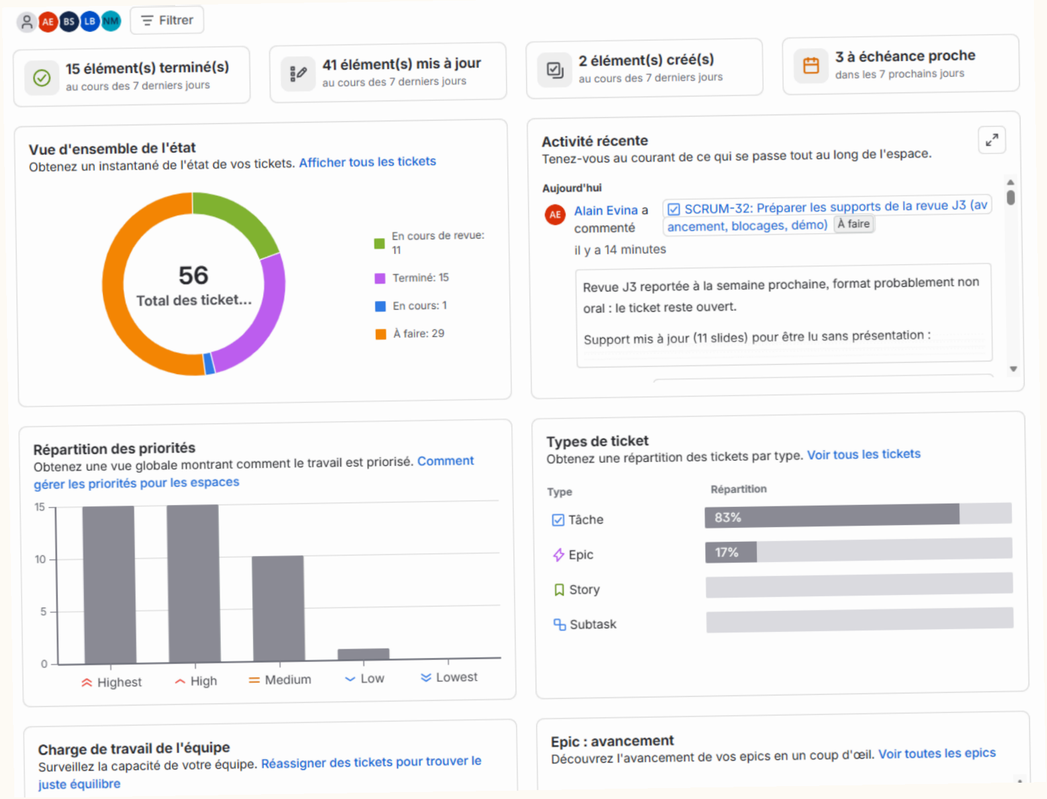
- [x] **Données, découpage et baseline**
  reproductibles, partagés par toute l'équipe
- [x] **BiLSTM v1 exporté en ONNX**
  parité validée, embarqué dans le site
- [-] **Site prêt, pas encore publié**
  reste la fusion de dev dans main pour GitHub Pages
- [-] **Adaptation LoRA engagée**
  chaîne validée sur CPU, essai GPU à lancer
- [x] **Jalon de mi-parcours manqué**
  preuves rassemblées, version taguée dès la mise en ligne
- [x] **Équipe réalignée**
  contrats v1.1, relecture des PR sous 48 h

## Ce qu'on a appris

Commencer par se mettre d'accord sur les formats. Ne jamais laisser une PR ouverte des jours. Mesurer avant d'affirmer.

*Le pilotage*

# Jira, tel qu'on l'utilise




**56**

tickets, chacun avec un responsable, un relecteur et un critère de fin

**15** terminés

**11** en revue

**1** en cours

**29** à faire

*41 tickets mis à jour en 7 jours*

*À vous de jouer*

# Le vérifier par soi-même

## Dans le navigateur

Coller des logs ou choisir un exemple, voir le score de chaque bloc. Le modèle est gardé en cache après le premier chargement.

```
lindaboukhit.github.io/TeamAI
```

*publication en cours*

## En local

Le SLM en version 4 bits sur une trace anormale : réponse structurée et mesures.

```
python -m volet_b.slm.bench_gguf
```

*commandes détaillées dans les README*

Code : github.com/LindaBOUKHIT/TeamAI, branche dev.

*Et maintenant*

# Sprint 2 : adapter, chercher, comparer

## Volet A

- Évaluer sur le test officiel
- Réexaminer le seuil de décision
- Parcours complet : lot, résultats par bloc, export CSV

## Volet B

- Jeu de test réservé et exemples relus
- Mesures « avant adaptation », sans et avec RAG
- LoRA complet, chaque essai consigné

### *Nos questions*

Quel périmètre attendez-vous pour l'adaptation ?
Sous quelle forme présenter la comparaison entre les deux volets ?

*merci !*

Alain, Nassim, Bachar et Linda